

Projet Semestriel 4A TL - S1

Intervenants du module

Matière	Intervenant
Framework Symfony	Baptiste Vasseur
Intégration Front - CSS Avancé & TailwindCSS	Édouard Sombié
IA intégrée à un projet web	Aloïs Marcellin
Automatisation des processus métier	Baptiste Vasseur
Versionning git avancé & pipelines CI/CD	Karl MARQUES BERNARDO
Performance web et optimisation	Edenn Touitou
Test d'applications (Unit, Intégration, e2e)	Sébastien Souphron
<u>Projet semestriel</u>	<u>Baptiste Vasseur</u>

Présentation du projet

Contexte

Le projet s'inscrit dans le cadre d'un module transversal visant à mobiliser un ensemble de compétences avancées en développement web full-stack. Il s'agit de concevoir, développer et déployer une application web e-commerce complète, permettant la vente de produits ou services en ligne.

Le projet servira de support à plusieurs enseignements techniques complémentaires : Symfony, intégration front (TailwindCSS), intelligence artificielle, automatisation, performance, tests, et CI/CD.

L'équipe devra produire un site e-commerce robuste, performant, maintenable, et intégrant des fonctionnalités modernes et intelligentes, tout en respectant les bonnes pratiques de versionning et de déploiement continu.

Objectif pédagogique

- **Mettre en œuvre un projet complet** de développement web intégrant un framework back-end (Symfony) et une intégration front avancée (TailwindCSS).
- **Approfondir la compréhension des processus CI/CD**, du versionning avancé avec Git, et de la mise en production automatisée.
- **Découvrir et intégrer de l'IA appliquée au web** (génération ou gestion de contenu, recommandation, automatisation).
- **Développer la capacité à auditer et optimiser les performances d'un site web**, en auditant le projet concurrent.
- **Appliquer les bonnes pratiques de test** (unitaires, d'intégration et end-to-end).
- **Développer une approche professionnelle de la gestion de projet** et de la documentation technique.

Liberté créative

Les étudiants disposent d'une **grande liberté dans le choix du concept** de la plateforme e-commerce :

- Type de produits ou services proposés (physiques, numériques, prestations, abonnements, etc.).
- Identité graphique et expérience utilisateur.
- Positionnement et ton du projet (boutique éthique, artisanale, tech, humoristique, etc.).
L'objectif est de favoriser la **créativité, l'autonomie et l'innovation**, tout en respectant les contraintes techniques et les exigences de qualité du code.

Enjeux techniques et professionnels

Le projet semestriel doit permettre aux étudiants de démontrer leur capacité à **concevoir et développer une solution logicielle complète**.

Les enjeux sont les suivants :

1. **Back-end** : utilisation du framework **Symfony**, création d'une API REST (et éventuellement MCP pour du tooling interne).
2. **Front-end** : intégration à l'aide de **TailwindCSS** et composants Twig réutilisables.
3. **IA intégrée** : automatisation de la génération de contenu (par exemple : fiches produits, recommandations, descriptions, tri intelligent, chatbot interne, etc.).
4. **Automatisation** : intégration possible de **bots (ex : Telegram)** pour gérer des actions automatisées (commandes, notifications, support client).
5. **Performance** : audit d'un site concurrent un mois avant la soutenance, puis mise en place d'optimisations back/front selon le rapport d'audit de l'équipe concurrente (caching, compression, lazy loading, CDN, etc.).

6. **CI/CD & Git** : mise en place d'un pipeline complet (tests, build, déploiement) et d'une gestion propre des branches/features.
7. **Tests** : mise en œuvre de tests unitaires, d'intégration et end-to-end (e2e).

Fonctionnalités principales (notées /15 pts)

1. **Gestion des utilisateurs**
 - Création de compte, authentification sécurisée.
 - Gestion des rôles, droits d'accès. (minimum client/gestionnaire/admin)
2. **Gestion des produits (CRUD complet, catégories, recherche, filtrage)**
 - Gestion des produits / catégories
 - Gestion des stocks
3. **Panier et système de paiement en ligne (Stripe, PayPal, etc.)**
 - Gestion des paiements
 - Gestion des remboursements (unitaire ou total)
4. **Interface d'administration pour la gestion du catalogue et des commandes**
 - Validation des commandes (si non automatisé)
 - Changement de statut des commandes (en préparation, en livraison, livré, remboursé)
 - Remise en stocks (annulation, remboursement partiel ou total)
5. **IA intégré**
 - Génération automatique de contenu produit (description / visuel / ...)
 - Recommandations personnalisées
 - Archivage de l'historique des demandes (côté client et prestataire).
6. **API REST sécurisé**
7. **Système d'automatisation**
 - Bot assistance et/ou gestion pimpé à l'IA (infos sur commande, gestion des commandes, demande de remboursement, gestion des stocks, ...)
 - Campagnes de mailing automatisées selon les événements utilisateurs (abandon de panier, nouvelle offre, feedback).
 - Personnalisation dynamique du contenu des emails via IA.

Fonctionnalités supplémentaires (notées /5 pts)

- **Marketplace complète**
 - Possibilité pour plusieurs marchands d'ajouter leurs propres services/produits.
 - Système de commission et reversement automatisé.
- **API MCP**
 - Création de tools (Agentic IA) pour manipuler la plateforme via l'API
 - Intégration dans le.s bot.s
- **Dashboard analytique**
 - Statistiques temps réel
 - Qualité de service (délais moyens, satisfaction, ...)

- **Tooling avancé**
 - Outils de diagnostic interne : analyse des logs, supervision des workflows langgraph.
 - Génération automatique de rapports de performance / erreurs.
 - Interface web dédiée au tooling interne.
 - Tooling via interface
 - Chat intégré avec generative IA
- ... (sous validation)

Liste non exhaustive, Chacune de ces fonctionnalités permet d'aller plus loin sur un plan technique et pro, et offre au jury un bon levier de différenciation entre les groupes.

Contraintes techniques et pédagogiques

Framework Symfony

- API (MCP pour du tooling et REST)

Intégration Front – CSS Avancé & TailwindCSS

- Bibliothèque de composants Twig gérés par tailwindCss

IA intégrée à un projet web

- 2 modèles différents minimum

Automatisation des processus métier

Versionning git avancé & pipelines CI/CD

- Mise en place d'une CI/CD complète du poste de travail au déploiement
- Sécurisation des branches (push/merge/review)
- Workflows séparés selon l'environnement (main => prod, release => preprod, branch => dev)
- Mise en place de tous les types de tests (linter, unit, functional, e2e)

Performance web et optimisation

- Audit de perf du site du concurrent à faire 1 mois avant la soutenance, mise en place des améliorations sur le dernier mois
- Optimisation des performances (back/front)

Test d'applications (Unit, Intégration, e2e)

Rendus attendus

Framework Symfony

Intégration Front – CSS Avancé & TailwindCSS

IA intégrée à un projet web

- Repository github du serveur de l'agent IA
- Rendu en .zip aussi
- Explication via PDF des prompts et pourquoi ils ont été fait ainsi
- Explication de la pipeline détaillée
- Présentation du front et de la générative UI

Automatisation des processus métier

Versionning git avancé & pipelines CI/CD

- Repository Github
- Invitation avec visibilité des actions (prévenir de l'invitation)

Performance web et optimisation

- dossier de l'audit de performances contenant
 - une analyse de l'application, quelle solution elle apporte à quel problème et quel.les utilisateur.ices
 - quels objectifs de performances doivent être choisis et quelles metrics sont donc à surveiller
 - quels outils et protocoles de tests sont à utiliser
 - résultats de ces protocoles de tests
 - conclusion des résultats et recommandations pour améliorer les performances
- lien du repository git avec une branche avant audit "before-audit" et une branche après audit et amélioration des performances

Test d'applications (Unit, Intégration, e2e)

- Sécuriser les endpoints d'API avec des tests d'intégration avec PHPUnit
- Rédiger des tests unitaires pour sécuriser les algorithmes côté back-end. Pour cela, utiliser dès que nécessaire des mocks.
- A l'aide du framework Cypress, écrire des tests end-to-end en particulier pour toutes les fonctionnalités principales de l'application.

Commun

- Repository GitHub (commits signés, README complet, répartition des tâches)
- Documentation technique
- Cahier des charges
- Mise en production
- Archive finale (code, doc, livrables)

Notation académique

L'évaluation académique des apprenants repose sur deux niveaux complémentaires :

1. Notation du module (contrôles continus)

L'objectif est de mesurer la progression et l'implication de l'étudiant **pendant le semestre** à travers un ou plusieurs contrôles continus (en accord avec le syllabus de cours).

Pondération : **40 % de la note finale académique.**

2. Notation du partiel du module

L'objectif est d'évaluer la capacité de l'étudiant à **réaliser un projet complet**, dans un contexte professionnel simulé, et à défendre son travail en soutenance ou via un rendu professionnel.

Pondération : **60 % de la note finale.**

3. Présentation collective – 20 minutes

L'objectif est de présenter **une seule fois** le projet final au jury.

- **Contenu attendu :**
 - Contexte du projet et problématique adressée.
 - Démonstration du produit / rendu final.
 - Mise en avant de la cohérence globale du projet.

- **Organisation :**
 - Le groupe choisit librement qui présente (tous les membres doivent prendre la parole).
 - La présentation doit être fluide, qualitative et concise.
- **Questions/réponses :**
 - **Questions techniques et méthodologiques** : approfondissement sur les choix, la mise en œuvre, les outils et les méthodes utilisés.
 - **Capacité de recul professionnel** : aptitude à justifier ses choix, proposer des alternatives, identifier les limites et perspectives.

4. Notation des partiels des matières associées

Chaque matière évaluera le projet en fonction des livrables qui lui sont propres pour déterminer sa note de partiel.